

How to write an amazing article

Adam Herout*

Abstract

What is the problem? What is the topic? The aim of this paper? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. How is the problem solved, the aim achieved (methodology)? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. Curabitur massa neque, commodo posuere fringilla ut, cursus at dui. Nulla quis purus a justo pellentesque. What are the specific results? How well is the problem solved? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. So what? How useful is this to Science and to the reader? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod.

Keywords: Keyword1 — Keyword2 — Keyword3

Supplementary Material: [Demonstation video](#) — [Downloadable code](#)

*herout@fit.vut.cz, Faculty of Information Technology, Brno University of Technology

1. Introduction

With the rise of Large Language Models (LLMs) and the increase in their capabilities, companies are motivated to use LLMs for automation tasks, including testing and the tooling for it. However, LLMs are not plug-and-play; specialized tooling is often required to produce reliable outputs. This work defines a formal base into which LLMs rewrite requirements and test cases written in natural language, so that the result is parsable by test coverage checking systems.

There is an increasing demand to instrument systems in natural language, including the specification of requirements and their test cases for testing systems and system components. Even when requirements and test cases are written in English, they must still be rewritten into a mathematical formal base so that existing test-coverage techniques can be applied. A formal base into which LLMs can convert human-written text is therefore needed. This formal base must be able to express all motivational requirement examples. Its semantics must be easily interpretable by LLMs and

agentic systems, so that agents can rewrite requirements and test case descriptions from human language into the formal base with high accuracy and without changing the semantics; the formalism itself must be unambiguous. It must also be convertible to existing formal bases used by formal verification tools. A good formalism should meet all of the mentioned criteria.

2. Existing methods of requirement formalisation – State of the Art

The translation from natural-language (NL) requirements to formal specifications has long been pursued through controlled-language frontends that constrain the input enough to make it unambiguous and further processable. The recent maturation of large language models has reopened the question of to what degree this translation can be automated without such restrictions, leading to LLM-based pipelines and hybrid approaches that pair LLMs with formal provers and solvers — the rest of this section surveys that landscape. This overview is informed by the recent short

survey of Beg et al. (2025) [1], which classifies the LLM-based literature in detail. It is further complemented with controlled-language frontends that the survey only briefly covers. Table 1 summarises the systems discussed in this section.

Controlled-language frontends. The longest-lived line of work attacks the problem at the *input* side by constraining the natural language itself. Nowakowski et al. (2013) [9] developed the *Requirements Specification Language* and the *ReDSeeDS* platform, which couples a constrained-NL requirements notation with automated transformations into code. Ghosh et al. (2016) [5] present *ARSENAL*, an NLP-based framework that extracts formal requirement specifications from natural language with automatic verification. NASA’s *FRET*, introduced by Giannakopoulou et al. (2020) [6], lets engineers write requirements in *FRETISH*, a restricted English with unambiguous semantics that maps deterministically to temporal logic.

LLM-only pipelines. With the rise of large language models, a growing body of work delegates the full translation to the LLM itself. Nayak et al. (2022) [8] present *Req2Spec*, an NLP pipeline that maps natural-language requirements onto the pattern-based formal specifications consumed by HANFOR; on 222 BOSCH automotive requirements it correctly formalises 71%. Cosler et al. (2023) [2] propose *nl2spec*, which uses few-shot prompting of Codex (code-davinci-002) and Bloom-176B to translate unstructured natural language into Linear-time Temporal Logic (LTL), with iterative user refinement of subformulas. Fang et al. (2024) [3] introduce *AssertLLM*, a three-stage pipeline of customised GPT-4o models (the final stage retrieval-augmented) that generates SystemVerilog Assertions from hardware design specifications with 89% correctness.

Hybrid LLM + prover systems. The most recent strand pairs an LLM with a formal prover or solver that has the final say on correctness. Jiang et al. (2022) [7] introduce *Thor*, which couples a 700M-parameter decoder-only transformer with the Isabelle/HOL theorem prover via Sledgehammer-based premise selection (“Hammers”), raising proof success on PISA from 39% to 57%. Yang et al. (2023) [11] release *LeanDojo* and the retrieval-augmented prover *ReProver*, a ByT5 encoder-decoder paired with premise retrieval over the Lean mathematics library and benchmarked on 98 734 theorems. Quan et al. (2024) [10] propose *ExplanationRefiner*, a refinement loop between an LLM (GPT-4, GPT-3.5-turbo, Llama2-70b, Mixtral-8x7b, or Mistral-small) and the Isabelle/HOL prover that validates and

Table 1. State-of-the-art systems for natural-language to formal-specification translation discussed in this section, in chronological order within each category. Abbreviations: HITL = human-in-the-loop; NLP = natural language processing; MDD = model-driven development; SAL = Symbolic Analysis Laboratory; LTL = linear-time temporal logic; HANFOR = an industrial requirements-analysis tool that consumes pattern-based formal specifications; SVA = SystemVerilog Assertions; HOL = higher-order logic (as in the Isabelle/HOL theorem prover); SMT = satisfiability modulo theories.

Ref.	System	Year	Models	Target	HITL
<i>Controlled-language frontends</i>					
[9]	RSL/ReDSeeDS	2013	— (rule-based)	Code (MDD)	Yes
[5]	ARSENAL	2016	NLP pipeline	SAL/LTL	No
[6]	FRET	2020	— (rule-based)	LTL	Yes
<i>LLM-only pipelines</i>					
[8]	Req2Spec	2022	NLP pipeline	HANFOR	No
[2]	nl2spec	2023	Codex, Bloom-176B	LTL	Yes
[3]	AssertLLM	2024	GPT-4o	SVA	No
<i>Hybrid LLM + prover systems</i>					
[7]	Thor	2022	700M trans-former	Isabelle/HOL	No
[11]	LeanDojo	2023	ByT5 + retrieval	Lean	No
[10]	ExplanationRefiner	2024	GPT-4/3.5, Llama2-70b, Mixtral-8x7b, Mistral-small	Isabelle/HOL	No
[4]	SAT-LLM	2024	LLM + SMT solver	SMT	No

corrects natural-language explanations. Fazelnia et al. (2024) [4] build *SAT-LLM*, which combines an LLM with an SMT solver to detect conflicting requirements at F1 0.91.

[[Add the gap and what is our contribution, basically summarize the next section]]

3. Contributing by designing a “close-to-NL” formalism

While prior controlled-language frontends [6, 9, 5] and LLM-based translators [2] target a single fixed logic — typically LTL or a tool-specific pattern language — none of the systems surveyed in Section 2 position their formalism around predicate-level matchability between requirements and test cases for coverage-gap detection, nor as a logic-agnostic intermediate explicitly designed to be the translation target for LLMs. The formalism presented in this section is designed around precisely these two properties.

109 3.1 Formalism requirements and its design

110 The formalism must satisfy the properties listed below.
111 Each property is paired with a concrete use case ex-
112 pressed in abstract placeholders (signals A, B ; events
113 e_A, e_B ; etc.) and a brief note on the construct the
114 formalism provides to address it.

115 **[[Fix ordering]]**

116 **[[Include a requirement and test case example,**
117 **so the reader can have a better understanding be-**
118 **fore reading the rest]]**

119 **Meta-level properties. Logic-agnostic representa-**
120 **tion of system behaviour in time.** The formalism
121 shall not commit to a specific underlying logic, so
122 that a single requirement representation can be retar-
123 geted to different verification back-ends. *Solution:* A
124 requirement is represented as a composition of two
125 kinds of building blocks: (i) ordinary expressions and
126 predicates that, when evaluated against a trace (the
127 same kind of object a test case produces), yield con-
128 crete values or truth values at each point in time; and
129 (ii) temporal predicates that aggregate those point-in-
130 time predicates into a single truth value over the entire
131 trace. The semantics of each building block is defined
132 directly in set-theoretic terms over traces and time,
133 independent of any target back-end. Every temporal
134 predicate has a direct LTL (or metric-LTL) equivalent
135 and a corresponding timed-automaton construction.

136 **Mechanical coverage-gap detection.** Given a set
137 of formalised requirements and a set of formalised test
138 cases, the formalism shall enable a checker to deter-
139 mine which requirement conditions are not exercised
140 by any test case. *Solution:* Requirements and test cases
141 are formalised over the same entities and the same
142 predicates, so that the same sub-formulas appear on
143 both sides. A checker can then evaluate, for each test
144 case, which meaningful combinations of sub-formulas
145 inside a requirement's formula are driven to true and
146 which to false, and from that compute the fraction of
147 relevant combinations that the test suite exercises.

148 **Extensible domains.** Real-world requirements
149 draw on an open-ended range of kinds of things in a
150 system (signals, registers, channels, states, and so on)
151 and of events that can happen to them (a register being
152 written, a channel transmitting a value, a state being
153 entered), and it is unrealistic to fix a closed catalogue
154 up front that covers every domain the formalism may
155 ever be applied to. The formalism shall therefore keep
156 these catalogues extensible. *Example:* To support a
157 new entity kind C representing network packets, only
158 the relevant catalogue is widened; the rest of the for-
159 malism is untouched. *Solution:* The set of entity kinds,
160 the set of value types, the assignment of permitted

attributes to each entity kind, and the assignment of 161
permitted events (parameterised by the entity they hap- 162
pen to) to each entity kind are all parameters of the 163
formalism rather than fixed sets. Adding a new entry 164
to any of them requires only registering it in the rele- 165
vant catalogue; existing constructs continue to work 166
unchanged. 167

Time and values. Discrete and continuous time. 168
The formalism shall support both discrete-time and 169
continuous-time requirements within a single vocabu- 170
lary. *Example:* A discrete-time requirement says “at 171
every step, A is incremented by 3”; a continuous-time 172
requirement says “ e_B must occur within 2 seconds of 173
 e_A ”. *Solution:* The time set T has two flavours: $T_D =$ 174
 $\mathbb{N} \cup \{\infty\}$ for discrete time and $T_C = \mathbb{R}_{\geq 0} \cup \{\infty\}$ for 175
continuous time. Each requirement carries a *flavour* $\in$ 176
 $\{\text{Discrete, Continuous}\}$ field that fixes T for its scope. 177

Entities of multiple kinds carry values, and op- 178
erations modify them in test cases. The formalism 179
shall represent entities of different kinds (registers, 180
states, communication channels, computed signals, 181
etc.) and let each carry a value at any given time 182
point. It shall also provide a way for test cases to de- 183
scribe how those values change over time. *Example:* 184
Register A , state B , and channel C each have a value 185
at every relevant time point; a test case may, for in- 186
stance, write a new value into A , transition B to a new 187
state, or transmit a value through C . *Solution:* The 188
formalism defines an extensible catalogue of entity 189
kinds (currently signal, storage, event, event-trigger, 190
channel, state, value, abstract, and set), and a valua- 191
tion maps each entity to its value at each time point of 192
a trace. Modifications of those values are expressed 193
through *operations* — an extensible catalogue of ac- 194
tions, currently including *calculate*, *write*, *read*, *fire*, 195
set_state, *transmit*, *receive*, *insert*, *remove*, and *clear* 196
— that a test case fires at specific time points to drive 197
the trace. Each entity kind has its own subset of appli- 198
cable operations (e.g. *write* applies to storage, *fire* to 199
event-triggers, *transmit* to channels). 200

Initial-value constraints. The formalism shall be 201
able to constrain the value of an entity at the initial 202
time point. *Example:* “Signal A has initial value 0.” *So-* 203
lution: The temporal construct $\text{Initial}(\psi) \Leftrightarrow \psi(p, 0)$ 204
asserts a point-in-time predicate at time 0. 205

Reference to an entity's value at another time 206
point. The formalism shall allow a constraint to refer 207
to the value an entity had (or will have) at an ear- 208
lier or later time point, enabling recurrences, toggles, 209
and look-ahead conditions. *Example:* “ $A(\text{Now}) =$ 210
 $A(\text{Now} - 1) + 3$ ”; “ B equals the negation of B 's previ- 211
ous value”; or “ C at the next step equals the current 212

213 value of D ". *Solution*: $Val_p(e, t)$ returns the value of
 214 e at any time t , past or future; for discrete time, $Prev$,
 215 $Next$, and $RelTime$ shift the time argument backward
 216 or forward; $ValBefore_p(e, t)$ exposes the left limit at t .

217 **Static entity-attribute access.** The formalism
 218 shall expose static attributes of an entity (independent
 219 of trace and time). *Example*: The address of register
 220 A is the constant $0xAA000018$ and can be referenced
 221 as such. *Solution*: Static attributes are stored in an en-
 222 tity's modifier map $\mu_e : ModKey \rightarrow ModVal$; accessors
 223 such as $Addr(e) = \mu_e(address)$ retrieve them without
 224 reference to p or t .

225 **Events and triggers. Event occurrence at a given**
 226 **time point.** The formalism shall express that an event
 227 occurs at a particular time point. *Example*: " e_A is hap-
 228 pening at t ." *Solution*: The predicate $Happening(e, t)$,
 229 where e is an entity of type `Event`.

230 **Past-event semantics.** The formalism shall ex-
 231 press that an event has occurred at some point up to
 232 the current time. *Example*: " e_A has happened at least
 233 once." *Solution*: The predicate $HasHappened_p(e, t)$.

234 **Reference to event-occurrence times.** The for-
 235 malism shall expose the time of a specific past or future
 236 occurrence of an event. *Example*: "the time of the most
 237 recent occurrence of e_A ." *Solution*: $LastOcc_p(ev, t, n)$
 238 returns the interval spanning the n most recent occur-
 239 rences of ev up to t ; $FirstOcc_p$ does the same in the
 240 forward direction.

241 **Counting event occurrences within an interval.**
 242 The formalism shall count how many times an event
 243 occurred within a given interval, with control over
 244 whether interval endpoints are included. *Example*:
 245 "the number of occurrences of e_A in the half-open in-
 246 terval $(t_1, t_2]$." *Solution*: $EvtOccCount_p(ev, i)$ returns
 247 the count.

248 **[[Rest is not reviewed]]**

249 **Conditional and logical structure. Value-conditional**
 250 **branches.** The formalism shall support constraints
 251 that branch on the current value of an entity. *Exam-*
 252 *ple*: "If A is true then B equals X ; otherwise B equals
 253 Y ." *Solution*: The comparison predicate Cmp and the
 254 implication predicate $Implies(A, B)$ together express
 255 value-guarded branches.

256 **Multi-condition guards.** The formalism shall
 257 combine conditions of different kinds (event happen-
 258 ings, value comparisons, occurrence counts, historical
 259 values) into a single guard. *Example*: "If e_A occurs
 260 and $B > 100$ and e_C has occurred at least twice, then
 261 D must follow." *Solution*: $Allof(C)$ takes a finite set
 262 of predicates of any kind and forms their conjunction;
 263 the formalism's predicates form a uniform space **Pred**,

so event-, value-, count-, and history-based predicates
 can all be conjoined.

Disjunctive triggers. The formalism shall express
 a consequence that follows from any one of several
 alternative triggers. *Example*: "If e_A or e_B or e_C occurs,
 then D must hold." *Solution*: $AnyOf(C)$ forms the
 disjunction of a finite set of predicates and may be
 used as the cause of a *Causes* construct.

Set-valued quantification. The formalism shall
 quantify over a finite set of entities and reason about
 properties of the subset that satisfies a predicate. *Ex-*
ample: "For every entity in $\{A, B, C, D\}$, predicate P
 holds"; or "at most two entities in $\{A, B, C, D\}$ have
 a value greater than 10." *Solution*: The Set entity
 kind together with the quantifiers $ForAll(S, P)$ and
 $Exists(S, P)$, plus *Filter* and *Size* for cardinality rea-
 soning over the subset that satisfies a predicate.

Temporal patterns. Sequencing. The formalism
 shall require multiple actions to occur in a specified
 order. *Example*: " e_A then e_B then e_C ." *Solution*:
 $Sequence(\psi_1, \dots, \psi_n) \Leftrightarrow \exists t_1 \leq t_2 \leq \dots \leq t_n : \psi_i(p, t_i) \text{ for all } i$.

Time-bounded causation. The formalism shall re-
 quire an effect to occur within a bounded duration of a
 cause. *Example*: "If e_A occurs, e_B must occur within 2
 seconds." *Solution*: $CausesWithin(\psi_{cond}, \psi_{effect}, d) \Leftrightarrow$
 $\forall t : \psi_{cond}(p, t) \Rightarrow \exists t' \in [t, t + d] : \psi_{effect}(p, t')$.

Sliding temporal windows. The formalism shall
 reason about a window of fixed length ending at the
 current time. *Example*: " e_A occurred at least twice
 in the last 10 seconds." *Solution*: An interval con-
 structor applied to *Now* produces the window, e.g.
 $MkIntervalOO(Now - d, Now)$, which can then be passed
 to $EvtOccCount_p$ or used as an argument to a *ForAll/Exists*
 over time.

Existence and global constraints. The formalism
 shall express both "at some point in the trace" and "at
 every time point in the trace". *Example*: "Eventually A
 equals 5"; " A is always non-negative". *Solution*: The
 temporal constructs $Eventually(\psi) \Leftrightarrow \exists t : \psi(p, t)$ and
 $Always(\psi) \Leftrightarrow \forall t : \psi(p, t)$.

[[Preferably without human in the loop]]

3.2 Formalism design decisions

[[Might already be covered by previous section]]

3.3 Formalism definition

**[[Add an appendix]] [[Add appendix with exam-
 ples]]**

310 **3.4 Known limitations and possible future ex-**
311 **tensions**

312 **[[Although this work claims that the formalism**
313 **should be close to english and is close in the sense**
314 **that it uses english words, the words themselves**
315 **might not appear in a similar order than they would**
316 **in english. This might be a limitation.]]**
317 **[[Add limitations from formalism document]]**

318 **3.5 Compatibility with well-known formal spec-**
319 **ification methods**

320 **3.6 Requirement and test case examples**

321 **3.7 Future work**

322 **4. Conclusions**

323 **Acknowledgements**

324 I would like to thank my supervisor Ing. Aleš Smrčka
325 Ph.D. for his help.

326 **References**

327 [1] BEG, A., O'DONOGHUE, D. and MONAHAN, R.
328 *A Short Survey on Formalising Software Require-*
329 *ments with Large Language Models*. 2025. Avail-
330 able at: [https://arxiv.org/abs/2506.](https://arxiv.org/abs/2506.11874)
331 [11874](https://arxiv.org/abs/2506.11874).

332 [2] COSLER, M., HAHN, C., MENDOZA, D.,
333 SCHMITT, F. and TRIPPEL, C. *Nl2spec: Inter-*
334 *actively Translating Unstructured Natural Lan-*
335 *guage to Temporal Logics with Large Language*
336 *Models*. 2023. Available at: [https://arxiv.](https://arxiv.org/abs/2303.04864)
337 [org/abs/2303.04864](https://arxiv.org/abs/2303.04864).

338 [3] FANG, W., LI, M., LI, M., YAN, Z., LIU, S.
339 et al. AssertLLM: Generating Hardware Verifi-
340 cation Assertions from Design Specifications via
341 Multi-LLMs. In: *2024 IEEE LLM Aided Design*
342 *Workshop (LAD)*. 2024, p. 1–1.

343 [4] FAZELNIA, M., MIRAKHORLI, M.
344 and BAGHERI, H. Translation Titans, Reasoning
345 Challenges: Satisfiability-Aided Language
346 Models for Detecting Conflicting Requirements.
347 In: *Proceedings of the 39th IEEE/ACM Inter-*
348 *national Conference on Automated Software*
349 *Engineering (ASE '24)*. New York, NY, USA:
350 Association for Computing Machinery, 2024,
351 p. 2294–2298.

352 [5] GHOSH, S., ELENIOUS, D., LI, W., LINCOLN,
353 P., SHANKAR, N. et al. ARSENAL: Auto-
354 matic Requirements Specification Extraction
355 from Natural Language. In: RAYADURGAM, S.
356 and TKACHUK, O., ed. *NASA Formal Methods*.

Cham: Springer International Publishing, 2016, 357
p. 41–46. 358

[6] GIANNAKOPOULOU, D., MAVRIDOU, A., 359
RHEIN, J., PRESSBURGER, T., SCHUMANN, J. 360
et al. Formal Requirements Elicitation with 361
FRET. In: *Joint Proceedings of REFSQ-2020* 362
Workshops, Doctoral Symposium, Live Studies 363
Track, and Poster Track. Pisa, Italy: [b.n.], 2020. 364

[7] JIANG, A. Q., LI, W., TWORKOWSKI, S., 365
CZECHOWSKI, K., ODRZYGÓDZ, T. et al. Thor: 366
Wielding Hammers to Integrate Language Mod- 367
els and Automated Theorem Provers. In: *Ad-* 368
vances in Neural Information Processing Sys- 369
tems. Curran Associates, Inc., 2022, p. 8360– 370
8373. 371

[8] NAYAK, A., TIMMAPATHINI, H. P., MURALI, 372
V., PONNALAGU, K., VENKOPARAO, V. G. 373
et al. Req2Spec: Transforming Software Re- 374
quirements into Formal Specifications Using Nat- 375
ural Language Processing. In: *Requirements* 376
Engineering: Foundation for Software Quality 377
(REFSQ 2022). Berlin, Heidelberg: Springer- 378
Verlag, 2022, p. 87–95. 379

[9] NOWAKOWSKI, W., ŚMIAŁEK, M., AM- 380
BROZIEWICZ, A. and STRASZAK, T. 381
Requirements-Level Language and Tools 382
for Capturing Software System Essence. *Com-* 383
puter Science and Information Systems. 2013, 384
vol. 10, no. 4, p. 1499–1524. 385

[10] QUAN, X., VALENTINO, M., DENNIS, L. A. 386
and FREITAS, A. *Verification and Refinement* 387
of Natural Language Explanations through LLM- 388
Symbolic Theorem Proving. 2024. Available at: 389
<https://arxiv.org/abs/2405.01379>. 390

[11] YANG, K., SWOPE, A., GU, A., CHALAMALA, 391
R., SONG, P. et al. LeanDojo: Theorem Prov- 392
ing with Retrieval-Augmented Language Models. 393
In: *Advances in Neural Information Processing* 394
Systems. Curran Associates, Inc., 2023, p. 21573– 395
21612. 396